



Sri Lanka Institute of Information Technology

Reverse Engineering and Runtime Manipulation of Legacy Game Software

Software Secure Systems - IE3042

Submitted by:

Registration Number	Name
IT23659230	LAKSHAN W M H C

Date of Submission:

17.04.2026

Table of Contents

1. INTRODUCTION	3
2. GAME SELECTION AND ENVIRONMENT SETUP	3
2.1 Game Selection	3
2.2 Environment Setup.....	4
3 STATIC AND DYNAMIC ANALYSIS	5
3.1 Static Analysis.....	5
3.2 Dynamic Analysis	6
3.3 Register and Memory Identification	6
3.4 Memory Search and Identification of Life Variable.....	7
4 ASSEMBLY LEVEL MODIFICATION	9
4.1 Instruction Identification.....	9
4.2 Modification Approach	10
4.3 Explanation	11
5 SECURITY ANALYSIS AND DISCUSSION.....	11
5.1 Missing Security Mechanisms	11
5.2 Mitigation Strategies	12
6 CONCLUSION	12

1. INTRODUCTION

Reverse engineering is the method of studying the internal design and execution of software without having the source code for that software. This topic is relevant to cybersecurity since reverse engineering helps analysts gain an insight into how software works and how vulnerabilities within software could be exploited.

Old software systems, particularly computer games that were built before, did not incorporate any form of security measures like anti-debugging, integrity checks, memory protection, and code obfuscation. Consequently, such applications are easier to analyze and modify during runtime..

In this project, the reverse engineering process of the game called **Dangerous Dave** for DOS will be carried out. The main purpose of this research was to detect where the life value of the player is saved and how it is processed throughout the game. Then, it was seen how this value is processed after a player's death. The required memory address and instructions regarding life processing in the game were examined using **DOSBox-X**.

2. GAME SELECTION AND ENVIRONMENT SETUP

2.1 Game Selection

For this particular assignment, the chosen game to be analyzed was **Dangerous Dave (DOS version)**. The choice of this game for our experiment is justified by the fact that it is an old offline one-person game which has rather simple runtime architecture.

Dangerous Dave was chosen for the following reasons:

- It is a legacy DOS game within the required time range.
- It runs locally without any online services.
- It stores game state values such as score, level, and lives in a predictable memory layout.
- It does not appear to implement modern protection techniques such as anti-debugging or memory validation.

2.2 Environment Setup

This was achieved using a **Windows** machine running **DOSBox-X** along with its debugger. DOSBox-X was employed to run the game under an emulated DOS environment, and the debugger was used for examining the registers, memory, and instructions executed when playing the game.

Tools Used:

- Windows operating system
- DOSBox-X
- DOSBox-X Debugger
- Dangerous Dave game executable

Setup Process:

1. DOSBox-X was launched.
2. The Dangerous Dave game directory was mounted as drive C.
3. The game files were listed using the DOS directory command.
4. The game executable was launched from the mounted directory.
5. The debugger was used during gameplay to inspect memory and disassembly.

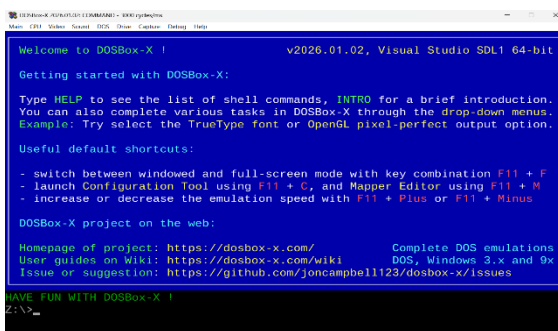


Figure 1 DOSBox-X startup interface

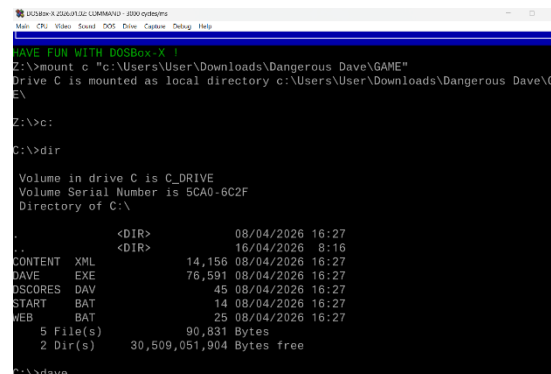


Figure 2 Mounting the Dangerous Dave directory and listing game files



Figure 3 Dangerous Dave title screen

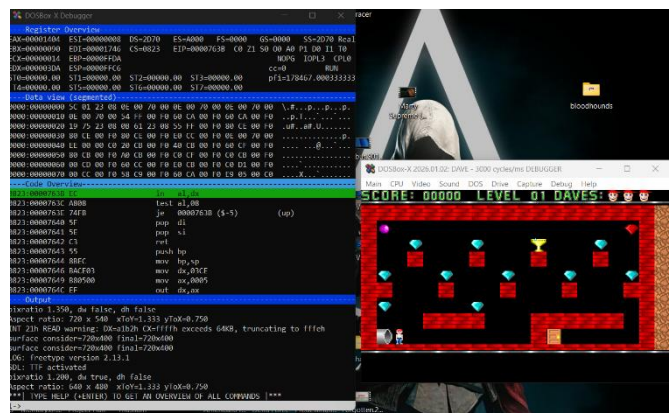


Figure 4 De-Debugger Window And Game

3 STATIC AND DYNAMIC ANALYSIS

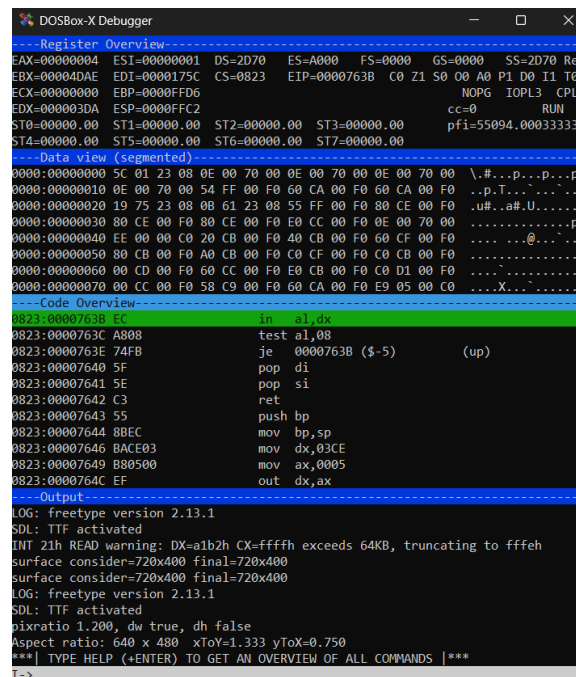
3.1 Static Analysis

After running the game, debugging was done by using the debugger interface to observe the program during runtime. Assembly instructions which constituted parts of the logic for the game were shown on the code window within the debugger. Among the instructions which could be observed included data transfer instructions, compare instructions, and branch instructions.

Examples of visible instructions included:

- **MOV** – used for moving data between memory and registers
- **CMP** – used for value comparison
- **JNE** / conditional jumps – used for branching based on logic
- Other supporting instructions related to runtime control flow

The code snippet obtained from the debugger indicated that the software was accessing memory addresses for the variables representing the state of the game directly. It became possible to associate memory addresses with behaviors in the game.



```
DOSBox-X Debugger
-----Register Overview-----
EAX=00000004  ESI=00000001  DS=2D70  ES=A000  FS=0000  GS=0000  SS=2D70  Rax
EBX=00004DAE  EDI=0000175C  CS=0823  EIP=0000763B  C0 Z1 S0 O0 A0 P1 D0 I1 T0
ECX=00000000  EBP=0000FFD6  NOPG  IOPL3  CPL0
EDX=00003DA  ESP=0000FFC2  cc=0  RUN
ST0=00000.00  ST1=00000.00  ST2=00000.00  ST3=00000.00  pfi=55094.00033333
ST4=00000.00  ST5=00000.00  ST6=00000.00  ST7=00000.00
-----Data view (segmented)-----
0000:00000000  5C 01 23 08 0E 00 70 00 0E 00 70 00 0E 00 70 00  \.#...p...p...
0000:00000010  0E 00 70 00 54 FF 00 F0 60 CA 00 F0 60 CA 00 F0  ..p.T.....
0000:00000020  19 75 23 08 0B 61 23 08 55 FF 00 F0 80 CE 00 F0  .u#..a#U.....
0000:00000030  80 CE 00 F0 80 CE 00 F0 E0 CC 00 F0 0E 00 70 00  .....p.....
0000:00000040  EE 00 00 C0 20 CB 00 F0 40 CB 00 F0 60 CF 00 F0  ....@.....
0000:00000050  80 CB 00 F0 A0 CB 00 F0 C0 CF 00 F0 C0 CB 00 F0  .....
0000:00000060  00 CD 00 F0 60 CC 00 F0 E0 CB 00 F0 C0 D1 00 F0  .....
0000:00000070  00 CC 00 F0 58 C9 00 F0 60 CA 00 F0 E9 05 00 C0  ....X.....
-----Code Overview-----
0823:0000763B  EC          in     al, dx
0823:0000763C  A808       test  al, 08
0823:0000763E  74FB       je     0000763B ($-5)      (up)
0823:00007640  5F         pop     di
0823:00007641  5E         pop     si
0823:00007642  C3         ret
0823:00007643  55         push   bp
0823:00007644  8BEC       mov     bp, sp
0823:00007646  BACE03     mov     dx, 03CE
0823:00007649  B80500     mov     ax, 0005
0823:0000764C  EF         out     dx, ax
-----Output-----
LOG: freetype version 2.13.1
SDL: TTF activated
INT 21h READ warning: DX=a1b2h CX=ffffh exceeds 64KB, truncating to fffeh
surface consider=720x400 final=720x400
surface consider=720x400 final=720x400
LOG: freetype version 2.13.1
SDL: TTF activated
pixratio 1.200, dw true, dh false
Aspect ratio: 640 x 480  xToY=1.333 yToX=0.750
***| TYPE HELP (+ENTER) TO GET AN OVERVIEW OF ALL COMMANDS |***
I->
```

Figure 5 Debugger view showing register, memory, and code sections

3.2 Dynamic Analysis

Dynamic analysis was performed by running the game normally and observing how the life counter changed during gameplay. When Dave was allowed to die by touching fire or another hazard, the visible number of lives decreased. This confirmed that the game updates the life variable in response to player death.

In the experiment, the game would be stopped right after the death event happens to use the debugger tool to analyze the program's current state. As such, it would be easier to determine what values were altered and what process was being executed during the change in the life value.

Prior to dying, there were three life icons that could be seen at the top right corner of the screen. After death occurred, there was an observation on the decrease in life icons.

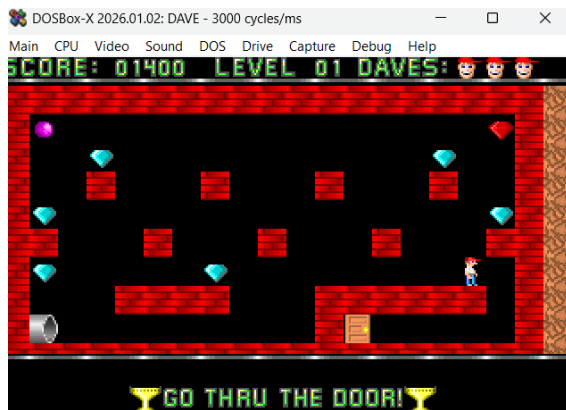


Figure 6 Game state before life reduction

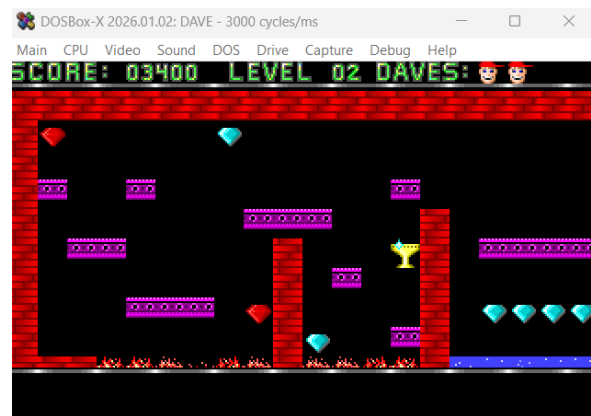


Figure 7 Game state after death event and life reduction

3.3 Register and Memory Identification

Registries were checked during debugging to determine which segment and what data were used in the update procedure. The **DS register** was especially important since it had data for the segment where all variables in the game existed.

The **DS register** value at the moment of debugging was **2D70**, and the data was in that segment's address. However, when analyzing the code, it was found that the program accessed data from the segment by offsetting them. In conclusion, it can be assumed that the variables such as the life counter existed in that segment.

Additionally, the debugger revealed that the program was using memory address **56EE**, which later proved to be the primary variable used for storing life.

```

DOSBox-X Debugger
-----Register Overview-----
EAX=00000004  ESI=00000001  DS=2D70  ES=A000  FS=0000  GS=0000  SS=2D70 Rea
EBX=00004DAE  EDI=0000175C  CS=0823  EIP=0000763B  C0 Z1 S0 00 A0 P1 D0 I1 T0
ECX=00000000  EBP=0000FFD6                                NOPG IOPL3 CPL0
EDX=000003DA  ESP=0000FFC2                                cc=0      RUN
ST0=00000.00  ST1=00000.00  ST2=00000.00  ST3=00000.00  pfi=55094.000333333
ST4=00000.00  ST5=00000.00  ST6=00000.00  ST7=00000.00

-----Data view (segmented)-----
0000:00000000  5C 01 23 08 0E 00 70 00 0E 00 70 00 0E 00 70 00  \. # . . . p . . . p . . .
0000:00000010  0E 00 70 00 54 FF 00 F0 60 CA 00 F0 60 CA 00 F0  .. p . T . . . . .
0000:00000020  19 75 23 08 0B 61 23 08 55 FF 00 F0 80 CE 00 F0  . u # . . a # . U . . . . .
0000:00000030  80 CE 00 F0 80 CE 00 F0 E0 CC 00 F0 0E 00 70 00  . . . . . p . . . . .
0000:00000040  EE 00 00 C0 20 CB 00 F0 40 CB 00 F0 60 CF 00 F0  . . . . . @ . . . . .
0000:00000050  80 CB 00 F0 A0 CB 00 F0 C0 CF 00 F0 C0 CB 00 F0  . . . . .
0000:00000060  00 CD 00 F0 60 CC 00 F0 E0 CB 00 F0 C0 D1 00 F0  . . . . .
0000:00000070  00 CC 00 F0 58 C9 00 F0 60 CA 00 F0 E9 05 00 C0  . . . . . X . . . . .

-----Code Overview-----
0823:0000763B EC          in     al, dx
0823:0000763C A808         test   al, 08
0823:0000763E 74FB         je     0000763B ($-5)      (up)
0823:00007640 5F          pop     di
0823:00007641 5E          pop     si
0823:00007642 C3          ret
0823:00007643 55          push    bp
0823:00007644 8BEC        mov     bp, sp
0823:00007646 BACE03      mov     dx, 03CE
0823:00007649 B80500      mov     ax, 0005
0823:0000764C EF          out     dx, ax

-----Output-----
LOG: freetype version 2.13.1
SDL: TTF activated
INT 21h READ warning: DX=a1b2h CX=ffffh exceeds 64KB, truncating to fffeh
surface consider=720x400 final=720x400
surface consider=720x400 final=720x400
LOG: freetype version 2.13.1
SDL: TTF activated
pixratio 1.200, dw true, dh false
Aspect ratio: 640 x 480  xToY=1.333 yToX=0.750
***| TYPE HELP (+ENTER) TO GET AN OVERVIEW OF ALL COMMANDS |***
I->

```

Figure 8 Register and memory observation during runtime analysis

3.4 Memory Search and Identification of Life Variable

To identify the exact memory location that stored Dave’s life value, a repeated memory search process was performed using the debugger’s memory search commands.

At the start of the search, Dave had **3 lives**, so the debugger was instructed to scan the data segment for occurrences of the value 03. This produced several candidate memory locations.

The following command sequence was used:

```

MEMFIND 2D70:0000 FFFF B
MEMS 3
MEMFIND L

```

After this initial result, the game was resumed and Dave was intentionally killed once so that the life count dropped from **3 to 2**. The memory search was then refined using:

```

MEMS 2
MEMFIND L

```

The process was repeated again after another life loss, when the value became **1**:

```

MEMS 1
MEMFIND L

```

By comparing the results across these repeated searches, it was observed that the address **2D70:56EE** consistently matched the visible life count and changed in the expected pattern:

- 03 when Dave had 3 lives
- 02 when Dave had 2 lives
- 01 when Dave had 1 life

This consistent behavior confirmed that **2D70:56EE** was the memory location associated with the game's life-related value.

```
---Output---
DEBUG: [MEMFIND] Address: 2D70:000112 Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:000255 Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:00053F Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:00058E Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:0005E3 Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:000680 Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:000685 Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:0006CF Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:000723 Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:000775 Current Value: 00000003
Paged content: Hit ENTER to continue, Q to exit paging
```

Figure 9 Initial memory search when Dave had 3 lives

```
---Output---
DEBUG: [MEMFIND] Address: 2D70:00A1C5 Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:00A1CE Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:00A1D7 Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:00A1D9 Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:00FE51 Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:00FF95 Current Value: 00000003
DEBUG: [MEMFIND] Matched values during current instance: 00007A
DEBUG: (000001) matches found with value EQUAL TO (000002). Iterations: 000002
DEBUG: [MEMFIND] Address: 2D70:0056EE Current Value: 00000002
DEBUG: [MEMFIND] Matched values during current instance: 000001
[->
```

Figure 10 Refined memory search when Dave had 2 lives

```
---Output---
DEBUG: [MEMFIND] Address: 2D70:00A1D9 Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:00FE51 Current Value: 00000003
DEBUG: [MEMFIND] Address: 2D70:00FF95 Current Value: 00000003
DEBUG: [MEMFIND] Matched values during current instance: 00007A
DEBUG: (000001) matches found with value EQUAL TO (000002). Iterations: 000002
DEBUG: [MEMFIND] Address: 2D70:0056EE Current Value: 00000002
DEBUG: [MEMFIND] Matched values during current instance: 000001
DEBUG: (000001) matches found with value EQUAL TO (000001). Iterations: 000003
DEBUG: [MEMFIND] Address: 2D70:0056EE Current Value: 00000001
DEBUG: [MEMFIND] Matched values during current instance: 000001
[->
```

Figure 11 Refined memory search when Dave had 1 life

To verify this result further, a **memory breakpoint** was placed on the identified location:

BPM 2D70:56EE

After resuming the game and triggering another life-related event, the debugger stopped automatically when the watched memory location was accessed or changed. This confirmed that the program was interacting directly with the identified life-related address during gameplay.

```
---Output---
surface consider=720x400 final=720x400
surface consider=720x400 final=720x400
LOG: freetype version 2.13.1
SDL: TTF activated
pixratio 1.200, dw true, dh false
Aspect ratio: 640 x 480 xToY=1.333 yToX=0.750
***| TYPE HELP (+ENTER) TO GET AN OVERVIEW OF ALL COMMANDS |***
DEBUG: Set memory breakpoint at 2D70:56EE
DEBUG: Memory breakpoint : 2D70:56EE - 00 -> 03
DEBUG: Memory breakpoint : 2D70:56EE - 03 -> 02
```

Figure 12 Memory breakpoint confirmation for the identified life variable

4 ASSEMBLY LEVEL MODIFICATION

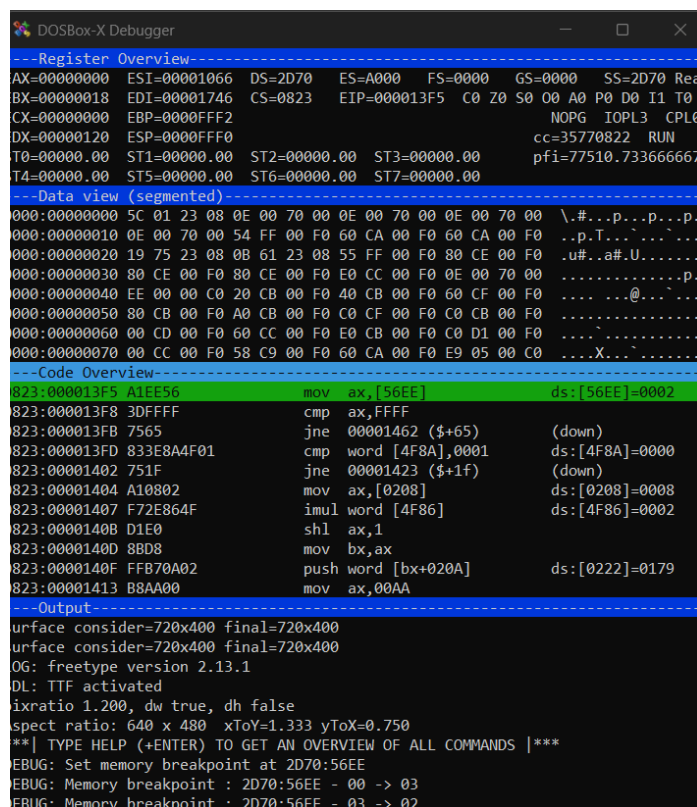
4.1 Instruction Identification

Once the memory breakpoint was placed on the confirmed life-related address **2D70:56EE**, the debugger stopped in a code region that directly referenced the same value. The code overview displayed the following instruction sequence:

```
0823:000013F5 mov ax,[56EE]
0823:000013F8 cmp ax,FFFF
0823:000013FB jne 00001462
```

It became clear from this set of instructions that the program was getting the value of **DS:[56EE]** and comparing/branching on it. Because of this fact that the watched address was already known to be the same one for Dave's number of lives from previous searches, this section was found to be responsible for dealing with life handling.

The use of this command, which states **"mov ax,[56EE]"**, is particularly significant as it verifies that the program makes reference to the life-related variable in the memory prior to further execution. From the following command sequence, one can conclude that the program evaluates the above-mentioned value and selects the path of future code execution based on its outcome. This instruction sequence was deemed to be the required assembly block.



```
DOSBox-X Debugger
---Register Overview---
AX=00000000 ESI=00001066 DS=2D70 ES=A000 FS=0000 GS=0000 SS=2D70 Rea
BX=00000018 EDI=00001746 CS=0823 EIP=000013F5 C0 Z0 S0 O0 A0 P0 D0 I1 T0
CX=00000000 EBP=0000FFF2             NOPG IOPL3 CPL0
DX=00000120 ESP=0000FFF0             cc=35770822 RUN
T0=00000.00 ST1=00000.00 ST2=00000.00 ST3=00000.00 pfi=77510.733666667
T4=00000.00 ST5=00000.00 ST6=00000.00 ST7=00000.00
---Data view (segmented)---
000:00000000 5C 01 23 08 0E 00 70 00 0E 00 70 00 0E 00 70 00 \. # . . . p . . . p . . .
000:00000010 0E 00 70 00 54 FF 00 F0 60 CA 00 F0 60 CA 00 F0 .. p . T . . . . .
000:00000020 19 75 23 08 0B 61 23 08 55 FF 00 F0 80 CE 00 F0 . u # . . a # . U . . . . .
000:00000030 80 CE 00 F0 80 CE 00 F0 E0 CC 00 F0 0E 00 70 00 . . . . . @ . . . . . p . . .
000:00000040 EE 00 00 C0 20 CB 00 F0 40 CB 00 F0 60 CF 00 F0 . . . . . @ . . . . .
000:00000050 80 CB 00 F0 A0 CB 00 F0 C0 CF 00 F0 C0 CB 00 F0 . . . . .
000:00000060 00 CD 00 F0 60 CC 00 F0 E0 CB 00 F0 C0 D1 00 F0 . . . . .
000:00000070 00 CC 00 F0 58 C9 00 F0 60 CA 00 F0 E9 05 00 C0 . . . . . X . . . . .
---Code Overview---
0823:000013F5 A1EE56      mov ax,[56EE]          ds:[56EE]=0002
0823:000013F8 3DFFFF      cmp ax,FFFF
0823:000013FB 7565      jne 00001462 ($+65)    (down)
0823:000013FD 833E8A4F01  cmp word [4F8A],0001  ds:[4F8A]=0000
0823:00001402 751F      jne 00001423 ($+1f)    (down)
0823:00001404 A10802      mov ax,[0208]         ds:[0208]=0008
0823:00001407 F72E864F  imul word [4F86]       ds:[4F86]=0002
0823:0000140B D1E0      shl ax,1
0823:0000140D 8BD8      mov bx,ax
0823:0000140F FFB70A02  push word [bx+020A]    ds:[0222]=0179
0823:00001413 B8AA00      mov ax,00AA
---Output---
urface consider=720x400 final=720x400
urface consider=720x400 final=720x400
OG: freetype version 2.13.1
DL: TTF activated
ixratio 1.200, dw true, dh false
spect ratio: 640 x 480 xToY=1.333 yToX=0.750
**| TYPE HELP (+ENTER) TO GET AN OVERVIEW OF ALL COMMANDS |***
EBUG: Set memory breakpoint at 2D70:56EE
EBUG: Memory breakpoint : 2D70:56EE - 00 -> 03
EBUG: Memory breakpoint : 2D70:56EE - 03 -> 02
```

Figure 13 Assembly code region accessing the life-related variable

4.2 Modification Approach

After identifying the relevant life-related address and the assembly region that accessed it, runtime intervention was performed using the DOSBox-X debugger. The debugger output confirmed that the watched memory location **2D70:56EE** changed when Dave lost a life, and the breakpoint mechanism made it possible to interrupt execution exactly when this value was being processed.

This illustrated that the game's life management mechanism could be affected while the program was running after locating the appropriate memory address. This is because the game stores the life count data in an easily identifiable region of the memory without integrity protection and anti-debugging, making it possible to manipulate its value when the game is running.

The main point of learning from this experiment is that the value of life assigned to the game character can be traced from gaming activity through the memory location to the assembly language code. The implication is that there is now a clear mechanism by which this information can be manipulated while the program is executing.

The experiment proved that the life mechanism used by the character "Dangerous Dave" is susceptible to manipulation during runtime because of the predictability of its memory allocation, as well as because of its accessibility.

```
---Output---
surface consider=720x400 final=720x400
surface consider=720x400 final=720x400
LOG: freetype version 2.13.1
SDL: TTF activated
pixratio 1.200, dw true, dh false
Aspect ratio: 640 x 480 xToY=1.333 yToX=0.750
***| TYPE HELP (+ENTER) TO GET AN OVERVIEW OF ALL COMMANDS |***
DEBUG: Set memory breakpoint at 2D70:56EE
DEBUG: Memory breakpoint : 2D70:56EE - 00 -> 03
DEBUG: Memory breakpoint : 2D70:56EE - 03 -> 02
I-> D 2D70:56EE
```

Figure 14 Memory Dumping

```
---Output---
LOG: freetype version 2.13.1
SDL: TTF activated
pixratio 1.200, dw true, dh false
Aspect ratio: 640 x 480 xToY=1.333 yToX=0.750
***| TYPE HELP (+ENTER) TO GET AN OVERVIEW OF ALL COMMANDS |***
DEBUG: Set memory breakpoint at 2D70:56EE
DEBUG: Memory breakpoint : 2D70:56EE - 00 -> 03
DEBUG: Memory breakpoint : 2D70:56EE - 03 -> 02
DEBUG: Set data overview to 2D70:56EE
DEBUG: Memory changed (2 bytes)
[=>
```

Figure 3 Giving the SM Command

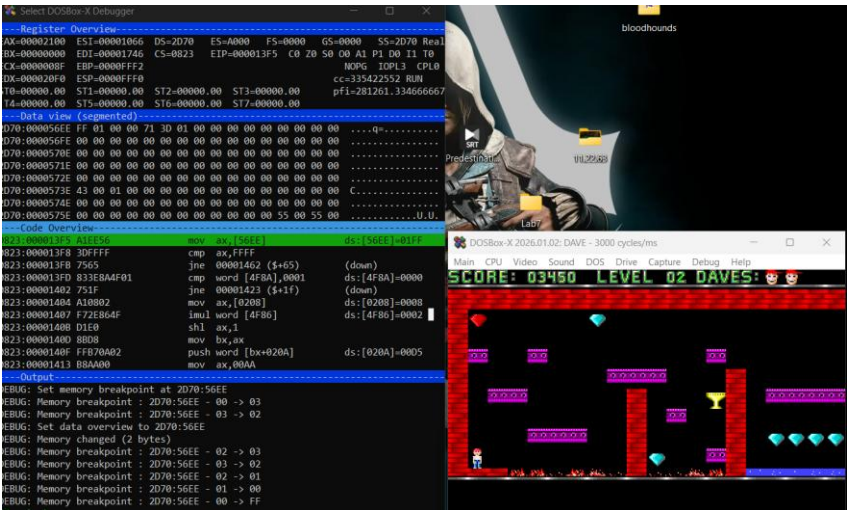


Figure 16 Runtime manipulation evidence for the watched life-related memory

4.3 Explanation

Manipulation of the runtime process is achievable because the game employs an address that clearly indicates the location where the lives data is stored. Once **2D70:56EE** was identified, debugging the program became quite straightforward through observing its access or modifications.

The intercepted code showed the program reading and changing the value stored in memory. Since there was no security mechanism to prevent observation and alteration, the program state was open to examination and alteration.

This is a classic example of a common flaw in old software: important game variables are kept unprotected in memory, thereby allowing for them to be found and modified by an analyst.

5 SECURITY ANALYSIS AND DISCUSSION

This attack is possible because the game lacks modern defensive mechanisms. The program allows direct runtime observation of its internal values and does not contain strong protections against reverse engineering or memory tampering.

5.1 Missing Security Mechanisms

The following missing protections made the experiment possible:

- **No anti-debugging**
The program did not detect or block execution under a debugger.
- **No memory protection**
Internal game-state values were exposed in memory and could be watched directly.
- **No integrity validation**
The software did not check whether important values had been modified during execution.
- **No code obfuscation**
The instruction flow remained readable enough for analysis and tracing.
- **Predictable data handling**
The life-related variable could be found through repeated value filtering.

5.2 Mitigation Strategies

Modern software systems reduce this kind of manipulation using several protections:

- **Anti-debugging techniques**
The software can detect debuggers and refuse to continue execution.
- **Integrity checks**
Important memory regions and code areas can be checked to detect tampering.
- **Memory protection mechanisms**
Critical values can be protected against unauthorized modification.
- **Code obfuscation**
The internal logic can be made harder to read and trace.
- **Randomized memory layout**
Variable locations can be made less predictable to attackers.
- **Validation of important state values**
The program can verify that values such as lives remain within expected limits and respond when abnormal changes are detected.

These protections make runtime analysis and manipulation significantly more difficult.

6 CONCLUSION

Reverse engineering the old game on the DOSBox-X debugger while playing the game happens in this manner. It involves recognizing the game event, filtering out memory values and setting a memory breakpoint that will give you the life value at **2D70:56EE**.

On closer inspection, it was discovered that the above code referenced this **memory location:**

mov ax, [56EE]

This showed that this memory address was used in the algorithm for managing life within the game. The debugger also showed that this watched memory could be altered during execution, proving that old software can be affected during runtime.

In general, from this experiment, one can note how insecure the old software systems were, lacking anti-debugging, integrity checking, and memory protection. This shows the significance of proper software security measures in current systems.

VIDEO LINK :- [IT23659230-SSS VIDEO.mkv](#)

-THANK YOU-